au fost 15 intrebari

1. calculeaza numarul de zerouri ale lui n!

[N/5] +[N/25]+[N/125]+[N/625]+...

2. pozitioneaza regine pe tabla de sah a.i. sa nu se atace

```
#include "stdio.h"
#include "conio.h"
#include "math.h"

int ataca(int linie, int memorie[])
//aceasta funcţie testează daca regina de pe linie  este atacata de
reginele poziţionate anterior. Este de fapt funcţia de Validare
{
      int i;
//luam pe rând toate damele poziţionate anterior
      for(i=0;i<linie;i++)
//verificam daca cele 2 dame sunt pe aceeaşi coloana sau pe diagonală
            if((memorie[linie]==memorie[i]) ||
                  (abs(memorie[i]-memorie[linie])==linie-i))
// se ataca, deci nu sunt corect poziţionate.
                        return 1;
      return 0; //daca am ajuns aici, înseamnă că nu se atacă
}



//căile de afişare sunt infinite
      void afiseaza(int dim,int memorie[])
      {
            int l,c;
            for(l=0;l<=dim;l++)
                  printf("_");
            for(l=0;l<dim;l++)
            {
                  printf("\n|");
                  for(c=0;c<dim;c++)
                        if(memorie[l]==c)
                              printf("%c",6);
                        else
                              if((l+c)&1)
                                    printf(" ");
                              else
                                    printf("%c",219);
                  printf("|");
            }
            printf("\n");
            for(l=0;l<=dim;l++)


                  printf("-");
            printf("\n");
            printf("o tasta, va rog!\n");
            getch();
      }



      void dame(int dim,int linie, int memorie[])
// aici este funcţia care face efectiv backtracking-ul (suntem la
// linia(nivelul) linie.
// dim ne spune cât de mare-i tabla
// memorie tine minte damele deja poziţionate. Ea este stiva
      {
        if(linie==dim)// daca am terminat
                  afiseaza(dim,memorie);//afişăm
        else //altfel avem de lucru
          for(memorie[linie]=0;memorie[linie]<dim;memorie[linie]++)
//încercam toate coloanele de la stânga la dreapta
                        if(!ataca(linie,memorie))
//daca e o poziţie valida trecem la nivelul următor
                              dame(dim,linie+1,memorie);
//când ne întoarcem din apelul recursiv înseamnă că am revenit
// la acest nivel şi încercăm următoarea coloana
      }

      void main()
      {
            int n,memorie[100];
            printf("dimensiunea tablei de joc=");
            scanf("%d",&n);
            dame(n,0,memorie);// prima linie are numărul zero
      }
```

```cpp
#include <cstdio>

#define MAX_N 14

int queen[MAX_N];
bool col[MAX_N], main_diag[MAX_N * 2], sec_diag[MAX_N * 2];
int ans;

void bkt( FILE *fout, int l, int n ) {
    if ( l == n ) {
        if ( ans < 1 ) {
            for ( int c = 0; c < n; c++ )
                fprintf( fout, "%d ", queen[c] + 1 );
            fprintf( fout, "\n" );
        }
        ++ans;
    } else {
        for ( int c = 0; c < n; c++ ) {
            if ( !col[c] && !main_diag[c - l + n - 1] && !sec_diag[l + c] ) {
                queen[l] = c;

                col[c] = main_diag[c - l + n - 1] = sec_diag[l + c] = true;
                bkt( fout, l + 1, n );
                col[c] = main_diag[c - l + n - 1] = sec_diag[l + c] = false;
            }
        }
    }
}

int main () {
    FILE *fin, *fout;

    fin = fopen( "damesah.in", "r" );
    int n;
    fscanf( fin, "%d", &n );
    fclose( fin );

    fout = fopen( "damesah.out", "w" );
    bkt( fout, 0, n );
    fprintf( fout, "%d\n", ans );
    fclose( fout );
}
```

3. algoritmul de fill din paint, considerand o imagine ca fiind o matrice cu elementele 0 si 1, si ti se da un punc tinterior din matrice, trebuie sa faci acea umplere

algoritm de umplere-FILL

Algoritmul de umplere- parcurge o zona continua cu 0-uri marginita de 1, dintr-o matrice, recursiv, prin umplerea acesteia cu o valoare diferita, de exemplu -1.
El porneste dintr-un punct (x,y) si incearca sa treaca in vecinii neparcursi ai acestuia, daca acestia se gasesc in interiorul matricei
La trecerea prin punctul (x,y) il marcheaza daca era nemarcat(il face -1)

```
#include<iostream>
#include<fstream>

using namespace std;
variabile globale
int n,m,a[100][100],nr;
nr-numarul de patrate din interiorul zonei

functia de umplere
void fill(int x,int y){
        daca patratelul nu a mai fost parcurs
        if(a[x][y]==0){
                nr++; il numara
                a[x][y]=-1;il marcheaza
                if(x>1)fill(x-1,y);incearca sa mearga mai sus
                if(y>1)fill(x,y-1);la stanga
                if(x<n)fill(x+1,y);mai jos
                if(y<m)fill(x,y+1);la dreapta
        }
}

afisarea matricii
void afisare(){
        for(int i=1;i<=n;i++){
                for(int j=1;j<=n;j++)
                        cout<<setw(4)<<a[i][j]<<" ";
                        cout<<endl;
                }
                cout<<endl;
        }

int main(){
        ifstream f("fill.in");
        f>>n>>m;
        for(int i=1;i<=n;i++)
                for(int j=1;j<=n;j++)
                        f>>a[i][j];
        pornim umplerea dintr-un punct din interiorul zonei
        fill(2,2);
        afisare();
        cout<<"Numarul de patrate="<<nr;
}
```

4. ai un vector neordonat, cu N elemente de la 1 la N - 1, si un duplicat
fara sa sortezi vectorul, gaseste duplicatul

```
#include <iostream>
#include <fstream>
#include <vector>
#include "vector_io.h"

using namespace std;

// partitionare facuta ca la carte :: cartea lui Cormen
int partition(vector<int> &v, int lower, int upper) {
	int pivot = v[upper];
	int i = lower - 1, j;
	for(j = lower; j < upper; j++)
	{
		if(v[j] < pivot) {
			i++;
			swap(v[i], v[j]);
		}
	}
	swap(v[i + 1], v[j]);
	return i + 1;
}

int find_missing(std::vector<int> v)
{
	int lower = 0, upper = v.size() - 1;
	while(lower < upper)
	{
		// partitionam si vedem
		int poz = partition(v, lower, upper);
		// daca pivotul se afla pe pozitia valoarea sa atunci inseamna ca
		// toate elementele mai mici decat pivot exista : deci cautam la
		// dreapta pivotului
		if(v[poz] == poz)
		{
			lower = poz + 1;
		}
		// altfel cautam in intervalul lower poz - 1 adica la stanga pivotului
		else if(v[poz] == poz + 1)
		{
			upper = poz - 1;
		}
	}
	// la final stim ca lower va fi indexul primului element care nu
	// respecta proprietatea v[x] == x, deci lower lipseste din vector

    return lower;
}

int main(void)
{
    std::ifstream f("date.in");
    std::vector<int> v1, v2;
    f >> v1;
    f >> v2;
    std::cout << find_missing(v1) << std::endl;
    std::cout << find_missing(v2) << std::endl;
    f.close();
}
```

5. algoritm de sortare cu complexitatea mai mica de n^2

```
mergesort(start, stop)
{
 if start<stop then
 {
  mijloc=(start+stop)/2
  mergesort(start, mijloc)  // sorteaza prima jumatate
  mergesort(mijloc+1, stop)  // sorteaza a doua jumatate
  merge(start, mijloc, stop)  // interclaseaza vectorii sortati
 }
}

merge(start, mijloc, stop)
{
 for i=start to stop
  b[i]=a[i] // copiaza jumatatile sortate intr-un vector auxiliar b

 i=start; j=mijloc+1; k=start;

 // copiaza inapoi in a pe cel mai mic dintre elementele curente in
 // cele 2 jumatati
 while (i<=mijloc) and (j<=stop)
  if b[i]<=b[j] then
   a[k++]=b[i++]
  else
   a[k++]=b[j++]

 // copiaza ce a mai ramas din prima jumatate, daca in ea a mai ramas
 while i<=mijloc
  a[k++]=b[i++]
 // similar pt a doua
 while j<=stop
  a[k++]=b[j++]
}
```

```
procedura QUICKSORT(A, inf, sup) este
| i← inf
| j← sup
| x← A[(i+j) div 2]
| repeta
| | cat timp (i<sup)/\ (A[i]<x) executa i← i+1
| | cat timp (j>inf) /\ (A[j]>x) executa j← j-1
| | daca i <=j atunci
| | |   t← A[i]; A[i]← A[j]; A(j)← t
| | |   i← i+1; j← j-1
| | |___■
| pana cand (i>j)
| daca (inf<j) atunci QUICKSORT(A, inf, j)
| daca (i<sup) atunci QUICKSORT(A, i, sup)
|___■
```

O posibilă implementare a algoritmului, in limbajul C sub forma unei funcții, este redată mai jos:

```c
void QUICKSORT(int inf, int sup) {
  int x, i, j, t;
  i = inf;
  j = sup;
  x = A[(i + j) / 2];
  do {
    while ( (i < sup) && (A[i] < x) ) i++;
    while ( (j > inf) &&(A[j] > x) ) j--;
    if ( i<= j ) {
      t = A[i];
      A[i] = A[j];
      A[j] = t;
      i++;
      j--;
    }
  } while ( i <= j );
  if ( inf < j ) QUICKSORT(inf, j);
  if ( i < sup ) QUICKSORT(i, sup);
}
```

6. BFS

```
BFS(s, G) {
    foreach (u ∈ V) {
        p(u) = null; // initializari
        dist(s,u) = inf;
        c(u) = alb;
    }
    dist(s) = 0; // distanta pana la sursa este 0
    c(s) = gri; //incepem prelucrarea nodului, deci culoarea
devine gri
    Q = (); //se foloseste o coada cu nodurile de prelucrat
    Q = Q + s; // adaugam sursa in coada
    while (!empty(Q)) { // cat timp mai am noduri de prelucrat
        u = top(Q); // se determina nodul din varful cozii
        foreach v ∈ succs(u) { // pentru toti vecinii
            if (c(v) = alb) {// nodul nu a fost gasit, nu e in
coada
                Ù// actualizam structura date
                dist(v) = dist(u) + 1;
                p(v) = u;
                c(v) = gri;
                Q = Q + v;
            } // close if
        } // close foreach
        c(u) = negru; //am terminat de prelucrat nodul curent
        Q = Q - u; //nodul este eliminat din coada
    } //close while
}
```

7. ai o matrice de numere in care un element este 0. Scrie un algoritm eficient care sa iti faca 0 pe linia si pe coloana pe care se gaseste elementul 0 (initial)

Aha, this is an old question.

7 down vote

1. Use one boolean variate(`isZeroInFirstRow`) saving if first row has zero element(s) or not and one boolean variate(`isZeroInFirstCol`) saving if first column has zero element(s) or not.
2. Then, traverse the whole matrix. If `cell(i,j)==0`, then set cell(0,j) and cell(i,0) to 0.
3. Traverse the first row of the matrix. If `cell(0,j)==0`, then set all elements in column(j) to 0.
4. Traverse the first column of the matrix. If `cell(i,0)==0`, then set all elements in row(i) to 0.
5. If `isZeroInFirstRow==true`, set all elements in row(0) to 0.
6. If `isZeroInFirstCol==true`, set all elements in column(0) to 0.

probleme de logica:

8. ai 1000 de sticle de vin, din care una e otravita. Otrava isi face efectul in 3 zile, ai 10 voluntari care sa incerce sa bea din fiecare sticla. Care e numarul minim de sticle pe care trebuie sa le arunci ca sa fi sigur ca ai eliminat-o pe cea otravita, daca ai 3 zile la dispozitie

Numerotati sticlele folosind sistemul binar(baza 2) pentru a afla numarul minim de sclavi.

De exemplu, pentru 8 sticle de vin avem nevoie de minim 3 sclavi: 8 = 2^3.

	Sticla 1	Sticla 2	Sticla 3	Sticla 4	Sticla 5	Sticla 6	Sticla 7	Sticla 8
Sclavul a	0	1	0	1	0	1	0	1
Sclavul b	0	0	1	1	0	0	1	1
Sclavul c	0	0	0	0	1	1	1	1

In acest fel, fiecare sclav o sa bea din sticla in dreptul careia apare 1. Daca mor toti 3, sticla otravita e 8. Daca mor primii 2, sticla otravita e 4.

Pentru a testa 1000 sticle e nevoie de minim 10 sclavi, cu care putem forma 1024 combinatii (2^10)

9. ai doua sticle, una de 7l una de 11l, cum faci sa masori 2l de apa

11-7-7 = 3
11 – 3 = 8
7 – 8 = 1

10. ai mai multe animale: vaci, porci, cai, si o oaie, si o configuratie a grajdurilor... apoi niste reguli.. trebuie sa vezi in ce grajd se gaseste oaia... credc ca o gasesti pe net...

11. ai n afirmatii de tipul : cel putin o airmatie e adevarata, cel putin doua afirmatii sunt adevarate... cel putin n afirmatii sunt adevarate... cum determini cate afirmatii sunt adevarate?
sau ceva in genul asta... cred ca o gasesti pe net

Cel putin una dintre cele 10 afirmatii este falsa.
Cel putin doua dintre cele 10 afirmatii sunt false.
Cel putin trei dintre cele 10 afirmatii sunt false.
Cel putin patru dintre cele 10 afirmatii sunt false.
Cel putin cinci dintre cele 10 afirmatii sunt false.
Cel putin sase dintre cele 10 afirmatii sunt false.
Cel putin sapte dintre cele 10 afirmatii sunt false.
Cel putin opt dintre cele 10 afirmatii sunt false.
Cel putin noua dintre cele 10 afirmatii sunt false.
Cel putin zece dintre cele 10 afirmatii sunt false.

Toate cele zece afirmatii se contrazic una pe alta. Prin urmare o singura afirmatie poate fi adevarata. Asa ca, exact noua afirmatii sunt false si doar afirmatia 9 e adevarata, cum bine ne-a deslusit si **Tone**.

12. ai un pod, si 100 de oameni vor sa il traverseze.unui om ii ia o ora sa traverseze de la un capat la celalalt. Daca doi oameni se intersecteaza, fiecare isi schimba sensul de mers. in cat timp vor traversa oamenii podul?

13. aduna doua numere reprezentate ca o lista inlantuita

Following are the steps.

1) Calculate sizes of given two linked lists.

2) If sizes are same, then calculate sum using recursion. Hold all nodes in recursion call stack till the rightmost node, calculate sum of rightmost nodes and forward carry to left side.

3) If size is not same, then follow below steps:

….**a)** Calculate difference of sizes of two linked lists. Let the difference be *diff*

….**b)** Move *diff* nodes ahead in the bigger linked list. Now use step 2 to calculate sum of smaller list and right sub-list (of same size) of larger list. Also, store the carry of this sum.
….**c)** Calculate sum of the carry (calculated in previous step) with the remaining left sub-list of larger list. Nodes of this sum are added at the beginning of sum list obtained previous step.
Follo

http://www.geeksforgeeks.org/sum-of-two-linked-lists/

Pseudocode:
Step 1. Traverse the linked lists and push the elements in two different stacks
Step 2. Pop the top elements from both the stacks
Step 3. Add the elements (+ any carry from previous additions) and store the carry in a temp variable
Step 4. Create a node with the sum and insert it into beginning of the result list

ex: 123 = 1->2->3
123 + 123 = 2 -> 4->6

14. sa faci swap intre doua valori, fara a folosi o variabila auxiliara

```
a = a + b;

b = a - b;

a = a - b;

//---------

if (a != b)
{
     a ^= b;
     b ^= a;
     a ^= b;
}


a^=b^=a^=b
```